APPROVAL DEMO REPORT

NDU Project RBAC Role Recommendation Report

Suggested role-based access model for the current application, based on the codebase feature surface, current access architecture, and current RBAC guidance from established platforms and security sources.

PREPARED FOR	PREPARED ON
NDU Project approval review	**April 7, 2026**

CURRENT APP PATTERN	RECOMMENDED MODEL
Single workspace owner + binary admin flag	**Layered system roles + workspace roles + scoped overlays**

This report is intended to support role approval first. Enforcement design can follow after approval.

1. Executive Summary

CORE WORKSPACE ROLES

7

Owner, PM, Lead, Contributor, Reviewer, Viewer, External

OPTIONAL SYSTEM ROLES

4

Platform Owner, Platform Admin, Billing Admin, Content Admin

CURRENT STATE

2

Admin vs non-admin, plus top-level owner fields

MAIN DESIGN RULE

Least privilege

Deny by default, validate every request, audit role changes

Recommendation: approve a layered model instead of adding more boolean flags. For this app, the most practical shape is **base workspace role + scope + optional specialist overlays**. That keeps the model understandable for demos now and enforceable later without role explosion.

The codebase already covers project, program, portfolio, planning, front-end planning, design, execution, procurement, contracts, vendors, security, SSHER, AI-assisted generation, integrations, billing, and a separate admin domain. A simple *admin/member* split is too coarse for that surface area.

The clean approval set for the workspace side is: **Workspace Owner, Project/Program Manager, Workstream Lead, Contributor, Reviewer/Approver, Viewer, and External Collaborator**. On the system side, the current single admin role should eventually be split into **Platform Owner, Platform Admin, Billing Admin, and Content Admin**.

2. What The Current Codebase Shows

Feature surface observed

- Project, program, and portfolio workspaces
- Initiation, business case, framework, planning, WBS, and FEP modules
- Design, technical alignment, architecture, tools integration, and execution modules
- Issue, risk, change, schedule, stakeholder, and progress tracking
- Procurement items, RFQs, purchase orders, contracts, vendor tracking, and cost data
- Security management, SSHER, access logs, and integration configuration
- Admin domain for users, projects, coupons, subscription lookup, and content management

Current access model in code

- **System access:** a user document carries `isAdmin` and `isActive` only.
- **Admin gate:** admin routes check `UserService.isCurrentUserAdmin()` and block everyone else.
- **Workspace ownership:** projects, programs, and portfolios are created with a single `ownerId`.
- **Workspace visibility:** list queries are primarily filtered by `ownerId`, not membership.
- **Security module:** the app already stores role, permission, setting, and access-log records per project.
- **Audit base:** project activity logging already exists and can support RBAC audit trails.

KEY CODE REFERENCES

`lib/models/user_model.dart:4` user model includes **isAdmin**.

`lib/services/user_service.dart:35` admin is derived from email/domain and Firestore.

`lib/screens/admin/admin_auth_wrapper.dart:30` admin domain gate checks host and admin status.

`lib/services/project_service.dart:628` projects are created with a single **ownerId**.

`lib/services/program_service.dart:9` programs are created with a single **ownerId**.

`lib/services/portfolio_service.dart:9` portfolios are created with a single **ownerId**.

`lib/screens/security_management_screen.dart:45` per-project security roles, permissions, settings, and logs are already stored.

`lib/services/activity_log_service.dart:68` activity logs already support audit trails.

Main architecture implication: the app does not yet have a shared workspace membership model. That means true RBAC enforcement should not be implemented as more flags on the user record. It needs object-scoped membership for **project**, **program**, and **portfolio** resources.

3. Research-Based Design Principles Used

- **Least privilege:** users should get only the minimum access needed for their job, both horizontally and vertically.
- **Deny by default:** any new resource or endpoint should be blocked until explicitly permitted.
- **Validate on every request:** UI hiding is not enough; backend and data-layer checks must also enforce policy.
- **Separate system roles from workspace roles:** global administration should not be mixed with day-to-day project management.
- **Use role + scope + relationship:** pure RBAC alone becomes brittle once you need object ownership, assigned records, or external sharing.
- **Keep very few owners:** high-privilege owner roles should be limited and narrowly scoped.
- **Support viewers and guests explicitly:** enterprise tools consistently distinguish editors from read-only and external users.
- **Log permission changes and sensitive access:** approval, billing, security, and membership changes should be auditable.

Why that matters here: NDU Project is not only a planning app. It also holds procurement data, contracts, vendor performance, security settings, integrations, and billing/admin operations. That mix makes coarse roles risky.

4. Recommended Role Model

4.1 Core workspace roles to approve now

Role	Typical user	What they should have access to	What they should not control
Workspace Owner	Project sponsor, accountable lead, business owner	Full access to one project/program/portfolio; membership and role assignment; scope, budget, procurement, design, execution, closure, reports, approvals, archive/delete, ownership transfer, full logs.	System-wide user admin, coupons, subscriptions, global content unless separately granted a system role.
Project/Program Manager	PM, delivery lead	Full operational control of planning, design coordination, execution, risks, issues, changes, meetings, stakeholders, reports, AI assistance, and	Ownership transfer, workspace deletion, global admin actions, final break-glass security overrides, billing admin.

Role	Typical user	What they should have access to	What they should not control
		most day-to-day workspace updates.	
Workstream Lead	Design lead, engineering lead, procurement lead, security lead, ops lead	Edit rights inside assigned workstream(s); approve or sign off within delegated module; manage assigned records, deliverables, and team inputs for that module.	Other workstreams outside scope, workspace membership, ownership, global admin, unrelated commercial or security areas unless explicitly scoped.
Contributor	Team member, analyst, coordinator	Create and update assigned records, draft sections, meeting notes, activities, issue and risk updates, progress entries, and permitted AI generations in assigned areas.	Approvals, role management, destructive settings, archive/delete, security administration, commercial approvals.
Reviewer / Approver	Sponsor delegate, governance reviewer, QA approver	Read everything in scope, comment, review, approve/reject gates, baselines, or change requests, and add approval notes.	General editing of day-to-day workspace content, membership changes, destructive actions, admin domain.
Viewer / Executive	Leadership, auditors, observers	Read-only dashboards, reports, exports, milestone status, logs, and approved documents.	Any edits, approvals, AI write actions, security changes, procurement editing, admin actions.
External Collaborator	Vendor, consultant, client reviewer	Only explicitly shared records or modules, typically procurement, RFQs, contract review packs, vendor responses, or selected status reports.	Internal planning, security settings, team data, admin tools, unshared workspace data, ownership or membership controls.

> **Recommended approval set:** if you want the cleanest first implementation target, approve these seven workspace roles now. They are broad enough for the whole app but still simple enough to explain and enforce.

4.2 Optional system roles to split the current admin role later

Role	Typical user	What they should have access to	What they should not control
Platform Owner	Founder, head of platform	All admin-domain functions, user lifecycle, global role policy, cross-workspace recovery access, coupons, subscription lookup, content editing, project oversight.	Should be very limited in count; do not assign casually.
Platform Admin	Operations or support admin	User administration, project oversight, support troubleshooting, read-all access where needed.	Billing/coupon authority and global content changes unless explicitly granted.
Billing Admin	Finance or revenue ops	Coupons, subscriptions, invoice/support lookup, billing-related reports.	User role administration, global content editing, workspace ownership changes.
Content Admin	Content or product operations	Inline app content editing and publishing.	User admin, billing admin, workspace deletion, security role changes.

This split is optional for phase 1, but it is the right long-term replacement for the current single `isAdmin` flag.

5. Workspace Module Access Matrix

A = administer | E = edit/create/delete in normal scope | P = approve/comment | R = read/export | S = assigned or delegated scope only | X = no access by default

Module / resource family	Owner	PM	Lead	Contributor	Reviewer	Viewer	External
Workspace settings, members, role assignment	A	E	X	X	X	R	X
Project/program/portfolio create, archive, delete, transfer	A	X	X	X	X	R	X

Module / resource family	Owner	PM	Lead	Contributor	Reviewer	Viewer	External
Initiation, business case, framework, planning, FEP	E	E	E/S	E/S	P	R	X
Design, architecture, technical alignment, deliverables	E	E	E/S	E/S	P	R	X
Execution, issues, risks, changes, schedule, meetings, progress	E	E	E/S	E/S	P	R	R/S
Commercial: procurement, RFQs, POs, contracts, vendors, cost	A	E	E/S	E/S	P	R	R/P/S
Security, SSHER, integrations, access settings	A	E	E/S	X	P	R	X
Approvals, baselines, sign-off, closure gates	A	E	P/S	X	P	R	X
Reports, dashboards, activity logs, audit views	R	R	R	R/S	R	R	R/S

Module / resource family	Owner	PM	Lead	Contributor	Reviewer	Viewer	External
AI generation and regeneration tools	E	E	E/S	E/S	R	R	X

Interpretation note: *Lead* and *Contributor* should almost always be scoped by module, record assignment, or delegated ownership. That is the main control that keeps the model from becoming too broad.

6. Role Detail Recommendations

Workspace Owner

- One accountable owner per workspace, with a hard limit of 2-3 total owners for safety.
- Can assign PMs, Leads, Contributors, Reviewers, and Viewers.
- Can approve high-risk actions: archive/delete, ownership transfer, final commercial approvals, security overrides.
- Should have full audit visibility across activity logs and security access logs.

Project / Program Manager

- Best default role for the person running delivery day to day.
- Should control planning, execution, coordination, AI drafting, reporting, and member assignment within the workspace.
- Should not control global admin, coupon/subscription actions, or ownership transfer.

Workstream Lead

- Use this for procurement lead, security lead, design lead, engineering lead, or ops lead.
- Give edit rights only for delegated modules and only where they are accountable.
- Allow module-level approval if you want distributed sign-off without giving full PM or Owner power.

Contributor

- Best for the majority of internal team members.
- Can update assigned records and draft content, but cannot approve, reassign roles, or change workspace settings.
- Useful for engineering, QA, analysts, coordinators, and operations contributors.

Reviewer / Approver

- Use for governance, sponsors, compliance reviewers, or QA sign-off users.
- Keep this role read-heavy and approval-focused to preserve separation of duties.

Viewer / Executive

- Essential for leadership and audit visibility.
- Should be explicitly read-only, including on AI actions and comments if you want a strict view role.

External Collaborator

- Do not treat external parties like normal members.
- Scope them to shared commercial records or status packs only.
- Default-deny internal planning, security, staffing, and admin data.

---

7. Recommended Enforcement Shape Later

For this app, enforcement should be built around **membership on a workspace object**, not only on the user profile. A practical structure would be:

- `users/{uid}` for account status and system roles.
- `projects/{projectId}/members/{uid}` for project role, module scopes, and overlays.
- `programs/{programId}/members/{uid}` and `portfolios/{portfolioId}/members/{uid}` for higher-level scopes.
- Optional overlays such as `commercial_admin`, `security_admin`, or `content_admin`.
- Relationship checks like `ownerId`, `createdById`, assigned owner, approver, or explicitly shared external record.

> **Recommended policy rule:** allow an action only when all three conditions are satisfied: **(1) role permits it**, **(2) scope covers the target workspace or module**, and **(3) any relationship condition is true** when required, such as record ownership, assignment, or approved external share.

Minimum controls that should exist once enforcement starts

- Deny by default for every write path.
- Validate permissions in UI, service layer, and Firestore rules or trusted backend paths.
- Log role changes, membership changes, approvals, billing actions, security changes, and denied access attempts.
- Separate destructive rights from editing rights.
- Keep a clear distinction between system roles and workspace roles.

---

8. Approval Recommendation

Approve these now

1. Workspace Owner
2. Project / Program Manager
3. Workstream Lead
4. Contributor
5. Reviewer / Approver
6. Viewer / Executive
7. External Collaborator

Approve these as the follow-on admin split

1. Platform Owner
2. Platform Admin
3. Billing Admin
4. Content Admin

Why this set is the right size: it is broad enough for the full app, small enough to explain in a demo, and compatible with a later enforcement pass without a redesign.

9. Sources

Codebase references reviewed

- `lib/models/user_model.dart:4` - system user model with **isAdmin** and **isActive**.
- `lib/services/user_service.dart:35` - admin derivation and user admin operations.
- `lib/screens/admin/admin_auth_wrapper.dart:30` - admin host gate and admin-only routing wrapper.
- `lib/services/project_service.dart:628` - project creation and owner-based visibility.
- `lib/services/program_service.dart:9` - program owner model.
- `lib/services/portfolio_service.dart:9` - portfolio owner model.
- `lib/routing/app_router.dart:28` - wide feature surface across planning, design, execution, security, SSHER, and admin domains.
- `lib/screens/security_management_screen.dart:45` - project security roles, permissions, settings, and logs.
- `lib/models/project_data_model.dart:2055` - role and permission data structures already exist in the model layer.
- `lib/services/activity_log_service.dart:68` - project activity log support.
- `lib/services/procurement_service.dart:18` - procurement, RFQ, purchase-order, and contract resource families.

- `lib/services/contract_service.dart:51` and `lib/services/vendor_service.dart:5` - commercial records include creator metadata and should be role-protected.
- `lib/services/tools_integration_service.dart:130` - integration/OAuth configuration is sensitive and should be restricted.
- `lib/services/subscription_service.dart:180` - billing/subscription operations are system-scope, not normal workspace scope.

External research references

- OWASP Authorization Cheat Sheet, accessed April 7, 2026: least privilege, deny by default, validate on every request, log authorization events, and prefer role + attribute/relationship checks for complex apps. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- Microsoft Learn, *Best practices for Azure RBAC*, updated March 30, 2025: grant only needed access, narrow scope, limit owners, and prefer group assignment. https://learn.microsoft.com/en-us/azure/role-based-access-control/best-practices
- Atlassian Support, *What are permission schemes in Jira?*: separate global permissions from space permissions and use space roles so admins can delegate per-space control without granting unwanted access. https://support.atlassian.com/jira-cloud-administration/docs/what-are-permission-schemes-in-jira/
- monday.com Support, *User types explained*, modified November 29, 2025: distinct admin, member, viewer, guest, and board-owner patterns support splitting full control from editing, read-only, and external access. https://support.monday.com/hc/en-us/articles/360002144900-User-types-explained